```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m23.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m23.c -o m23
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m23
==========================================
          COMMAND HISTORY BUFFER
==========================================

Commands:
  add <command>  - Store command
  history        - Display history
  get <number>   - Retrieve command
  clear          - Clear history
  exit           - Exit

history-shell> hi
Invalid command.

history-shell> add
Invalid command.

history-shell> clear
History cleared.

history-shell>
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
#define MAX_COMMAND 256

typedef struct
{
    char commands[MAX_HISTORY][MAX_COMMAND];
    int count;
    int start;
} History;

void init_history(History *history)
{
    history->count = 0;
    history->start = 0;
}

void add_command(History *history, const char *command)
{
    int position;

    if (history->count < MAX_HISTORY)
    {
        position =
            (history->start + history->count) %
            MAX_HISTORY;

        strcpy(history->commands[position], command);

        history->count++;
    }
    else
    {
        position = history->start;

        strcpy(history->commands[position], command);

        history->start =
            (history->start + 1) %
            MAX_HISTORY;
    }
```

```
            (history->start + 1) %
            MAX_HISTORY;
    }
}

void display_history(const History *history)
{
    int i;
    int position;

    if (history->count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    printf("\n============================================\n");
    printf("              COMMAND HISTORY\n");
    printf("============================================\n");

    for (i = 0; i < history->count; i++)
    {
        position =
            (history->start + i) %
            MAX_HISTORY;

        printf("%d  %s\n",
               i + 1,
               history->commands[position]);
    }
}

const char *get_history_entry(
    const History *history,
    int number)
{
    int position;

    if (number < 1 || number > history->count)
```

```
const char *get_history_entry(
    const History *history,
    int number)
{
    int position;

    if (number < 1 || number > history->count)
        return NULL;

    position =
        (history->start + number - 1) %
        MAX_HISTORY;

    return history->commands[position];
}

int main()
{
    History history;

    char input[MAX_COMMAND];

    int number;

    init_history(&history);

    printf("============================================\n");
    printf("          COMMAND HISTORY BUFFER\n");
    printf("============================================\n");

    printf("\nCommands:\n");
    printf("  add <command>  - Store command\n");
    printf("  history        - Display history\n");
    printf("  get <number>   - Retrieve command\n");
    printf("  clear          - Clear history\n");
    printf("  exit           - Exit\n");

    while (1)
    {
        printf("\nhistory-shell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
```

```
        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strcmp(input, "history") == 0)
        {
            display_history(&history);
        }
        else if (strcmp(input, "clear") == 0)
        {
            init_history(&history);
            printf("History cleared.\n");
        }
        else if (strncmp(input, "add ", 4) == 0)
        {
            char *command = input + 4;

            if (strlen(command) == 0)
            {
                printf("Cannot store empty command.\n");
            }
            else
            {
                add_command(&history, command);
                printf("Command stored successfully.\n");
            }
        }
        else if (strncmp(input, "get ", 4) == 0)
        {
            number = atoi(input + 4);

            const char *command =
                get_history_entry(&history, number);

            if (command == NULL)
            {
```

```
            if (strlen(command) == 0)
            {
                printf("Cannot store empty command.\n");
            }
            else
            {
                add_command(&history, command);
                printf("Command stored successfully.\n");
            }
        }
        else if (strncmp(input, "get ", 4) == 0)
        {
            number = atoi(input + 4);

            const char *command =
                get_history_entry(&history, number);

            if (command == NULL)
            {
                printf("Invalid history entry.\n");
            }
            else
            {
                printf("Entry %d: %s\n",
                       number,
                       command);
            }
        }
        else if (strlen(input) == 0)
        {
            continue;
        }
        else
        {
            printf("Invalid command.\n");
        }
    }

    printf("\nHistory buffer consistency verified.\n");

    return 0;
}
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m24.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m24.c -o m24
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m24
==========================================
       PIPELINE COMMAND EXECUTION
==========================================

Examples:
ls -l | wc -l
ps aux | grep bash
cat /etc/passwd | grep root | wc -l

pipeline-shell> l

==========================================
             PIPELINE LAYOUT
==========================================
Process 1: l

Executing pipeline...
execvp: No such file or directory

pipeline-shell> m1

==========================================
             PIPELINE LAYOUT
==========================================
Process 1: m1

Executing pipeline...
execvp: No such file or directory

pipeline-shell> m1.c

==========================================
             PIPELINE LAYOUT
==========================================
Process 1: m1.c

Executing pipeline...
execvp: No such file or directory

pipeline-shell>
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ cat m24.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 1024

typedef struct
{
    char command[MAX_INPUT];
    char *args[MAX_ARGS];
    int argc;
} Command;

typedef struct
{
    Command commands[MAX_COMMANDS];
    int count;
} Pipeline;

void parse_command(char *input, Command *cmd)
{
    int i = 0;

    char *token =
        strtok(input, " \t\n");

    while (token != NULL &&
           i < MAX_ARGS - 1)
    {
        cmd->args[i++] = token;

        token =
            strtok(NULL, " \t\n");
    }

    cmd->args[i] = NULL;
    cmd->argc = i;
}

void parse_pipeline(char *input, Pipeline *pipeline)
```

```
    char *command_text;

    pipeline->count = 0;

    command_text =
        strtok(input, "|");

    while (command_text != NULL &&
           pipeline->count < MAX_COMMANDS)
    {
        parse_command(
            command_text,
            &pipeline->commands[pipeline->count]
        );

        pipeline->count++;

        command_text =
            strtok(NULL, "|");
    }
}

void display_pipeline(const Pipeline *pipeline)
{
    int i;
    int j;

    printf("\n==========================================\n");
    printf("             PIPELINE LAYOUT\n");
    printf("==========================================\n");

    for (i = 0; i < pipeline->count; i++)
    {
        printf("Process %d: ",
               i + 1);

        for (j = 0;
             j < pipeline->commands[i].argc;
             j++)
        {
            printf("%s ",
                   pipeline->commands[i].args[j]);
        }

        printf("\n");
    }
}
```

```
void execute_pipeline(Pipeline *pipeline)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    int i;

    if (pipeline->count == 0)
        return;

    /*
     * Create pipes
     */
    for (i = 0; i < pipeline->count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return;
        }
    }

    /*
     * Create processes
     */
    for (i = 0; i < pipeline->count; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return;
        }

        if (pids[i] == 0)
        {
            /*
             * Connect input from previous pipe
             */
            if (i > 0)
            {
                dup2(
                    pipes[i - 1][0],
                    STDIN_FILENO
                );
            }
```

```
            dup2(
                pipes[i - 1][0],
                STDIN_FILENO
            );
        }

        /*
         * Connect output to next pipe
         */
        if (i < pipeline->count - 1)
        {
            dup2(
                pipes[i][1],
                STDOUT_FILENO
            );
        }

        /*
         * Close all pipe descriptors
         */
        for (int j = 0;
             j < pipeline->count - 1;
             j++)
        {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }

        /*
         * Execute command
         */
        execvp(
            pipeline->commands[i].args[0],
            pipeline->commands[i].args
        );

        perror("execvp");
        exit(EXIT_FAILURE);
    }
}

/*
 * Parent closes all pipes
 */
for (i = 0;
     i < pipeline->count - 1;
     i++)
```

```
    }

    /*
     * Parent closes all pipes
     */
    for (i = 0;
         i < pipeline->count - 1;
         i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    /*
     * Wait for all processes
     */
    for (i = 0;
         i < pipeline->count;
         i++)
    {
        waitpid(
            pids[i],
            NULL,
            0
        );
    }
}

int main()
{
    char input[MAX_INPUT];

    Pipeline pipeline;

    printf("================================================\n");
    printf("       PIPELINE COMMAND EXECUTION\n");
    printf("================================================\n");

    printf("\nExamples:\n");
    printf("ls -l | wc -l\n");
    printf("ps aux | grep bash\n");
    printf("cat /etc/passwd | grep root | wc -l\n");

    while (1)
    {
        printf("\npipeline-shell> ");
        fflush(stdout);
```

```
    printf("ls -l | wc -l\n");
    printf("ps aux | grep bash\n");
    printf("cat /etc/passwd | grep root | wc -l\n");

    while (1)
    {
        printf("\npipeline-shell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        parse_pipeline(
            input,
            &pipeline
        );

        if (pipeline.count == 0)
        {
            printf("Invalid pipeline.\n");
            continue;
        }

        display_pipeline(&pipeline);

        printf("\nExecuting pipeline...\n");

        execute_pipeline(&pipeline);
    }

    printf("\nPipeline execution completed.\n");

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```